Software Engineer Assignment from Actual Inc (Indonesia)

Objective: Build a full-stack web application from a given brief, integrate a real AI image API, and **prove it works** even when things go wrong. We aren't looking for impressive engineering, we're looking for engineers who ship clean, reliable software on top of systems they didn't build, and who can architect scalable systems from scratch.

Step 1: What You're Building

Pick a specific “niche” for your image generation app: **architecture**, **manga**, **interior design**, **fashion**, **game concept art**, or anything else you find interesting. Build an AI image generation web app with a personal gallery

- **The Goal:** Build something a real user in that niche could open and use without any explanation
- **The Gallery:** Generated images and their prompts must be saved and visible. The gallery must still be there after a page refresh.
- **Re-generation:** Pick any saved result, tweak the prompt, and re-generate, without starting over.

Required Constraints (Non-Negotiable)

- AI API calls go through your backend only, never the browser.
- Images must be stored server-side. The gallery must persist across page refreshes.
- The app must work correctly with multiple users generating at the same time.
- Loading must be meaningful: **10 to 30 seconds is normal.**
- Handle failure states e.g. API timeout, invalid prompt, and a broken response.
- The app must be live via a public URL at submission.
- Use any free AI image API. Pollinations.ai and Gemini Flash Image API are good starting points.
- **AI coding assistants are allowed**, but you must understand what you submit. We'll ask you to walk through your code.

Step 2: The Thinking

Before writing a single line of code, document your system design. This will be read before we look at anything else.

- **App Journey:** How does a user's prompt travel from the browser, through your backend, to the AI API, and back as an image?

  Reference Flow (Example):

  1. User inputs text and/or an image on the frontend
  2. The frontend sends the input to your backend
  3. The backend passes the input to the AI image API
  4. The AI API returns the result to the backend
  5. The backend responds to the frontend with the generated image
  6. The user sees the result on the frontend
- **Tech Stack:** What stack did you choose and why?
- **Your Process:** What did you prioritize first and why? How did you sequence the work?

Step 3: The Execution

Build the app. The implementation is fully your choice. Make sure you follow the **non-negotiable part** on **Step 1**

- **Frontend:** Usable without instructions. The loading experience, gallery, re-generation flow, and all three error states must be handled visibly.
- **Backend:** Handles all AI API calls, stores images, and manages the gallery. Handle the "messy" cases: slow API, broken response, concurrent users.
- **AI Knowledge:** Your implementation should reflect an understanding of how AI image generation works: its “inputs”, “outputs”, “limitations”, and “failure modes”.
- **"Craft":** We will read the code, not just run it. Clear names, straightforward logic, no unexplained shortcuts. Commit as you go, one large commit is a red flag.
- **README:** Anyone should be able to run the app locally in under 15 minutes. Cover setup, environment variables, and how to start the app.

Step 4: The Proof

Show us the app working with real inputs and real API responses. No fake data, no pre-recorded demos, no scripted flows that skip the hard parts.

- **The Full Flow:** Record a Loom showing the full journey: prompt, wait, image, save, re-generate. Show a live API call. Do not skip the wait.
- **The Failures:** Show at least two of the three required failure states on camera. Demonstrating failure is not optional. It is the most important part of this step.
- **The Honest Part:** Write down at least one thing that still doesn't work well. An honest observation beats a claim that everything works perfectly.

Step 5: Submit

1. **The Repo Link:** GitHub or GitLab. README must get us running in under 15 minutes.
2. **The Documentation:** A single document (PDF or Google Doc) covering:
   - **Your Thinking:** Request journey, tech stack choice, and build process from Step 2.
   - **Your Decisions:** The key technical choices you made and why.
3. **The Demo Link:** Loom or screen recording showing the full flow, gallery, re-generation, and at least two failure states.
4. **The Live URL:** Your deployed app, fully working at review time.

Scoring Criteria

Category	Weight	Score (1-5)	What We Look For
Level of Thinking	25%		• Did they plan before they built? • Is the request journey clear and specific? • Is the tech stack choice deliberate and justified? • Does the process show intentional planning? • Do they understand the tools and systems they used, or did they just use them?
Level of Execution	35%		• Is the front end usable without instructions? • Does the gallery persist across refreshes? • Does re-generation work cleanly? • Is the backend clean, readable, and correct under concurrent load? • Does the README work in under 15 minutes? • Do the commits show real, step-by-step work?
Data and Validation	20%		• Is the live URL fully working, not just reachable? • Does the recording show real API responses, not mocked data? • Are at least two failure states demonstrated on camera? • Does the candidate understand where their app falls short?
Problem Solving	20%		• Did the niche shape the product, or is it just a label? • Did they handle complexity without over-engineering? • Can they explain what they chose not to build and why? • Does the implementation show real understanding of how AI image generation works?

Logistics and Criteria

- Deadline: 7 days after receiving and accepting this assignment. If you focus, this should take 7 to 8 hours of actual work.
- Budget: If you need to use a paid service, we will reimburse up to $25. Save your receipts.
- Submission: Email your repo link, live URL, demo recording, and all documents to dic@actu-al.co, chm@actu-al.co, and peb@actu-al.co.
- Integrity: We can tell the difference between a repository that was genuinely worked through and one that was generated and pushed all at once. We value the "messy" truth of a real build over a suspiciously clean result.
- Keep it Simple: No slide decks. Give us the live URL, the recording, and the thinking behind it.

Good luck and have fun.